CODE SPORT

Sandya Mannarswamy

In this month's column, we will continue our discussion of compiler optimisations, and focus on compiler analyses and optimisations that require inter-procedural analysis and inter-procedural code transformation.

In last month's column, we had posed a coding snippet (shown below), and had asked our readers how it can be rewritten to reduce data cache misses associated with that code region.

```
int** two_dim_array =  (int*) calloc (sizeof (int*) * N);
for (int j = 0; j < N; j++)
    two_dim_array[j] = calloc(sizeof(int) * N);
….. // read in input values for the elements of two_dim_
array[i][j]
for (int i = 0; i < N; i++)
for (int j = 0; j < N; j++)
  result   + =  Two_dim_array[i][j];
```

Thanks to all those who sent in their solutions to this question. Congrats to our reader Prashanth Shah for getting the correct solution. The problem with the above code snippet is that the adjacent rows of the array are not contiguous, while the elements of a single row itself are contiguous. For instance, if we assume N as 100, then the elements *two_dim_array[0][98]* and *two_dim_array[0][99]* are contiguous, but they are not contiguous to *two_dim_array[1][0]*, the first element of the next row. Hence, whenever there is a new iteration of the outer *'for'* loop (*'i'* in the code above), it can lead to a compulsory cache miss. Therefore, in order to avoid this cache miss, and enable optimisations like data pre-fetching, which can read-ahead the next element being accessed, we need to change the allocation pattern of the original array. Instead of allocating each row independently, the compiler can perform an optimisation known as *'malloc* combine transformation', wherein it combines the '*calloc'* calls in Line 1 and Line 3 of the code snippet above, and allocates a two-dimensional array of size *two_dim_array[N][N]* contiguously, as shown below:

```
int* temp =  (int*) calloc (sizeof (int) * N *N);
for (int j = 0; j < N; j++)
    two_dim_array[j] = &temp[j][0];
….. // read in input values for the elements of two_dim_
array[i][j]
for (int i = 0; i < N; i++)
for (int j = 0; j < N; j++)
  result   + =  Two_dim_array[i][j];
```

Inter-procedural analysis and optimisation

Now, let us turn our attention to compiler analysis, and the optimisation that requires the compiler to look at code occurring in more than one function—and, in many cases, in all the functions that are present in the application. Typically, the compiler is invoked on a *.c* file that contains code for multiple functions. An application consists of multiple *.c* files, each of which contains code for multiple functions. Inter-procedural analysis and optimisation can happen at both the intra-file and the inter-file level. Can you think of a very common compiler optimisation that requires the compiler to examine more than one function to perform?

Well, the answer is the optimisation known as inlining, where the compiler replaces the call site in a function with the complete code of the called function. For instance, if the function '*foo'*